Greedy Algorithms

US Change Problem

United States Change Problem:
Convert some amount of money into the fewest number of coins.

Input: An amount of money, M, in cents.

Output: The smallest number of quarters q, dimes d, nickels n, and pennies p whose values add to M (i.e., $25q + 10d + 5n + p = M$ and $q + d + n + p$ is as small as possible).

```
BETTERCHANGE(M, c, d)
1  r ← M
2  for k ← 1 to d
3        i_k ← r/c_k
4        r ← r − c_k · i_k
5  return (i_1, i_2, ..., i_d)
```

It is a **greedy algorithm:**

At every step of iteration, a greedy algorithm tries to find the best optimal solution (e.g., used the most the coin with the biggest value)

US Change Problem

United States Change Problem:
Convert some amount of money into the fewest number of coins.

Input: An amount of money, M, in cents.

Output: The smallest number of quarters q, dimes d, nickels n, and pennies p whose values add to M (i.e., $25q + 10d + 5n + p = M$ and $q + d + n + p$ is as small as possible).

```
BETTERCHANGE(M, c, d)
1  r ← M
2  for k ← 1 to d
3        i_k ← r/c_k
4        r ← r − c_k · i_k
5  return (i_1, i_2, ..., i_d)
```

Why **greedy?**

"greedy" means having excessive desire for something without considering the effect or damage done.

US Change Problem

```
BETTERCHANGE(M, c, d)
1  r ← M
2  for k ← 1 to d
3        i_k ← r/c_k
4        r ← r − c_k · i_k
5  return (i_1, i_2, ..., i_d)
```

Does it always find a correct solution?

When c_1 = 25, c_2 = 20, c_3 = 10, c_4 = 5, c_5 = 1,

if M = 40, BetterChange returns i_1 = 1, i_3 = 1, i_4 = 1

We would solve the problem with i_2 = 2…

US Change Problem

```
BETTERCHANGE(M, c, d)
1  r ← M
2  for k ← 1 to d
3        i_k ← r/c_k
4        r ← r − c_k · i_k
5  return (i_1, i_2, ..., i_d)
```

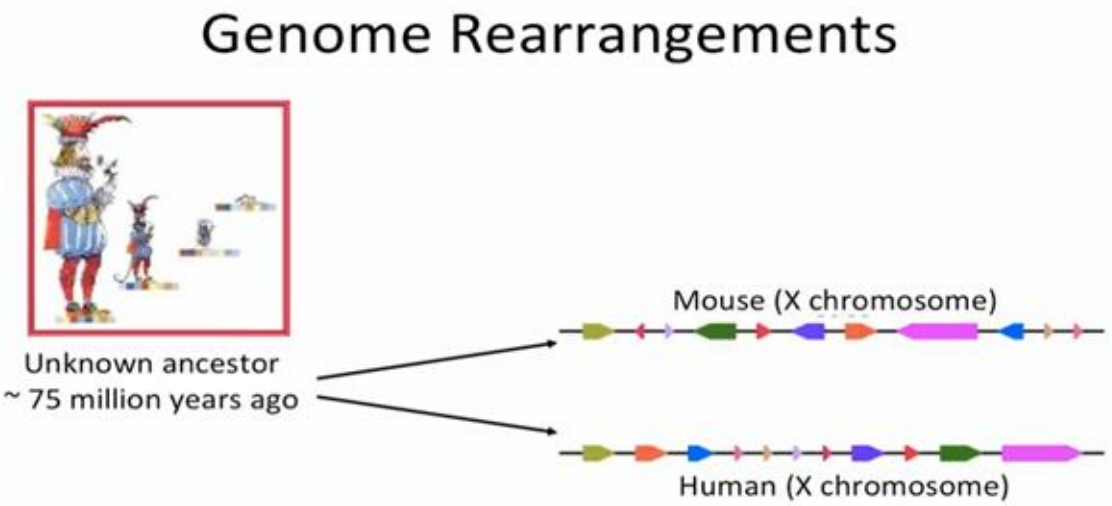
We can ask ourselves: how close are we from the optimal solution?

Maybe this algorithm works almost always correctly

Outline of the lecture

We are going to see a **bioinformatic problem** that we try to solve with **different greedy algorithms** and we are going to **evaluate their goodness** in finding the **best solution**

Let us see a problem in Biology



- What are the similarity blocks and how to find them?
- What is the evolutionary scenario for transforming one genome into the other?

https://www.youtube.com/watch?v=lCoUp2Bq8OA&list=PLQ-85lQlPqFOcGz6A3g2ZArRL09Ffpp_N (until 8:52)

Permutations (def)

- Permutation: an ordering of the integers 1, …, *n* in which each number appears **exactly once**
- Examples (*n* = 5):

  π = 3 1 5 2 4 σ = 2 5 4 1 3
- Not permutations:

  3 1 3 2 4 (3 repeated, 5 missing)

  3 1 6 2 4 (6 > *n*)
- There are *n*! permutations of *n* elements (*n* = 5 → 120)
- In genomes, each number is a gene and the permutation is their order along the chromosome

Reversal Distance Problem

- Goal: Given two permutations, find the shortest series of reversals that transforms one into another
- Input: Permutations $\boldsymbol{\pi}$ and $\boldsymbol{\sigma}$
- Output: A series of reversals $\rho_1,\ldots\rho_t$ transforming $\boldsymbol{\pi}$ into $\boldsymbol{\sigma}$, such that t is minimum
- $\boldsymbol{t}$ - reversal distance between π and σ
- $\boldsymbol{d}(\pi, \sigma)$ - smallest possible value of t, given π and σ

Sorting By Reversals Problem

- Goal: Given a permutation (i.e., a vector in a random order), find a shortest series of reversals that transforms it into the identity permutation (*1 2 … n*)
- Input: Permutation $\boldsymbol{\pi}$
- Output: A series of reversals $\rho_1, \ldots \rho_t$ transforming $\boldsymbol{\pi}$ into the identity permutation such that *t* is minimum

Reversals

- Reversal ρ (*i, j*) reverses (flips) the elements from *i* to *j* in π

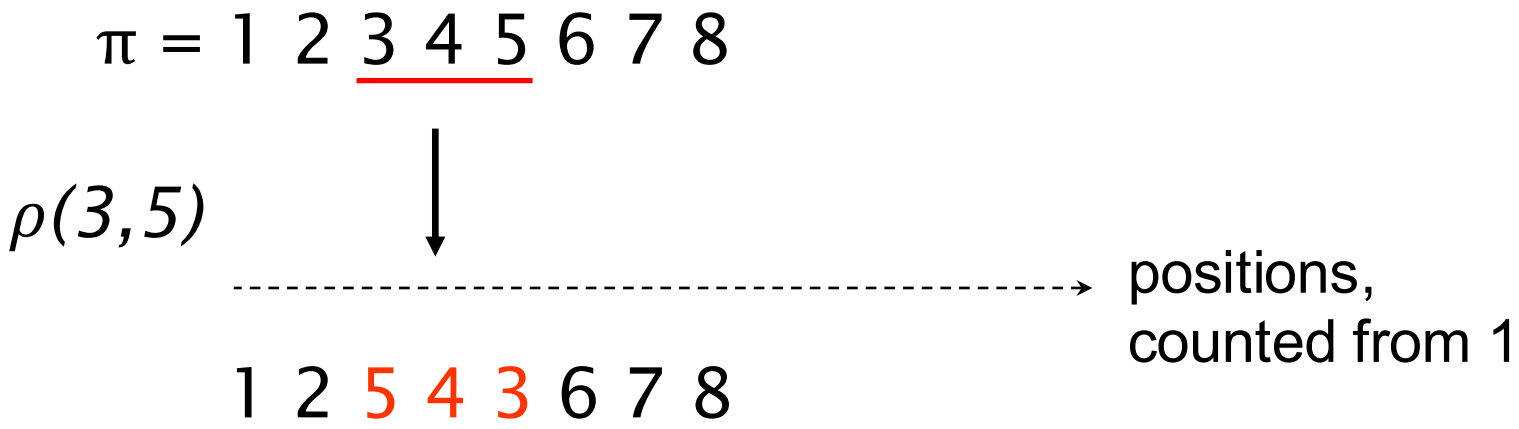
$$\pi = \pi_1 \text{ ----- } \pi_{i-1}\ \pi_i\ \pi_{i+1} \text{ ----- } \pi_{j-1}\ \pi_j\ \pi_{j+1} \text{ ---- } \pi_n$$

$$\rho(i,j)$$

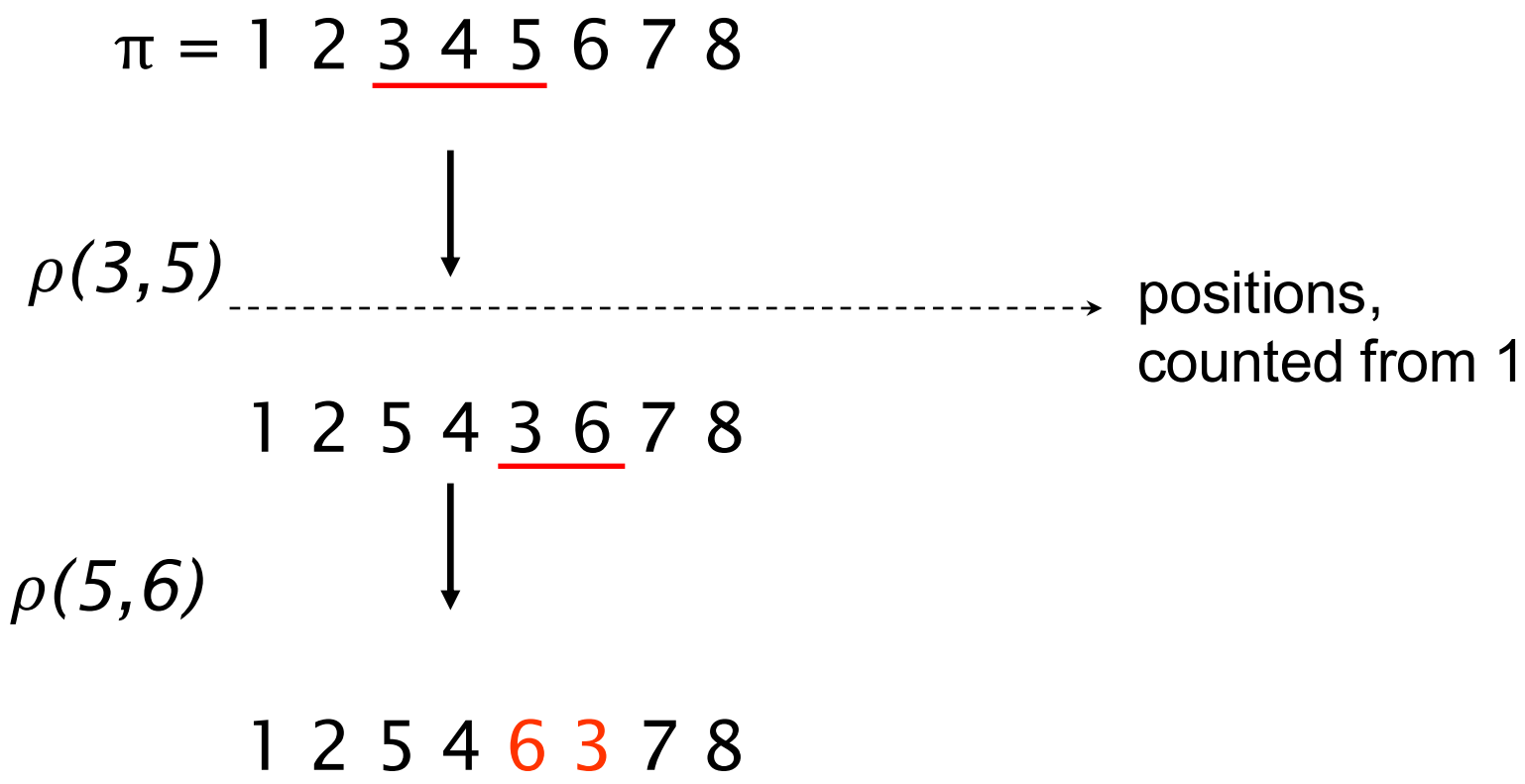
$$\pi_1 \text{ ----- } \pi_{i-1}\ \pi_j\ \pi_{j-1} \text{ ----- } \pi_{i+1}\ \pi_i\ \pi_{j+1} \text{ ---- } \pi_n$$

Note: positions start at 1, so π_1 is the first element and π_n the last

Reversals: Example



Reversals: Example



Sorting By Reversals: Example

- $t = d(\pi)$ - reversal distance of π
- Example :

π = 3 4 2 1 5 6 7 10 9 8

4 3 2 1 5 6 7 10 9 8

4 3 2 1 5 6 7 8 9 10

1 2 3 4 5 6 7 8 9 10

So $d(\pi) = 3$

Sorting by reversals: 5 steps

Step 0: π	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	2	3	4	5	6	7	8	1
Step 3:	2	3	4	5	6	7	8	-1
Step 4:	-8	-7	-6	-5	-4	-3	-2	-1
Step 5: γ	1	2	3	4	5	6	7	8

Sorting by reversals: 4 steps

Step 0: π	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	-5	-4	-3	-2	-8	-7	-6	1
Step 3:	-5	-4	-3	-2	-1	6	7	8
Step 4: γ	1	2	3	4	5	6	7	8

Sorting by reversals: 4 steps

Step 0: π	2	-4	-3	5	-8	-7	-6	1
Step 1:	2	3	4	5	-8	-7	-6	1
Step 2:	-5	-4	-3	-2	-8	-7	-6	1
Step 3:	-5	-4	-3	-2	-1	6	7	8
Step 4: γ	1	2	3	4	5	6	7	8

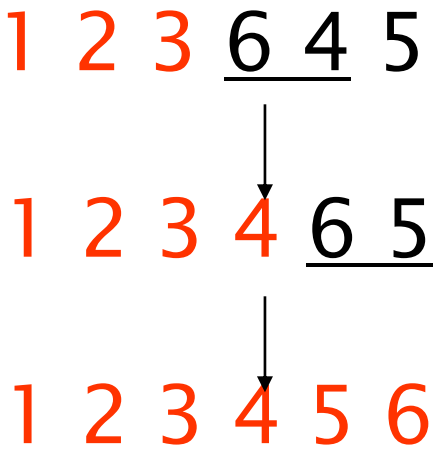
What is the reversal distance for this permutation? Can it be sorted in 3 steps?

Sorting By Reversals: A Greedy Algorithm

- If sorting permutation π = 1 2 3 6 4 5, the first three elements are already in order so it does not make any sense to break them.
- The length of the already sorted prefix of π is denoted *prefix*(π)
  - *prefix*(π) = 3
- This results in an idea for a greedy algorithm: ***increase prefix(π) at every step***

Greedy Algorithm: An Example

- Doing so, π can be sorted



- Number of steps to sort permutation of length *n* is at most *(n – 1)*

Greedy Algorithm: Pseudocode

SimpleReversalSort(π)

```
1 for  i ← 1 to n - 1  # assuming that elements are from 1 to n
2    j ← position of element i in π (i.e., π_j = i)
3    if  j ≠ i
4       π ← we apply ρ(i, j) on π
5    if π is the identity permutation
6      return π
```

Analyzing SimpleReversalSort

- SimpleReversalSort does not guarantee the smallest number of reversals and takes five steps on π = 6 1 2 3 4 5 :

Step 0: 6 1 2 3 4 5

- Step 1: 1 6 2 3 4 5
- Step 2: 1 2 6 3 4 5
- Step 3: 1 2 3 6 4 5
- Step 4: 1 2 3 4 6 5

Analyzing SimpleReversalSort (cont'd)

- But it can be sorted in two steps:

  π = 6 1 2 3 4 5

  - Step 1: 5 4 3 2 1 6
  - Step 2: 1 2 3 4 5 6
- So, SimpleReversalSort(π) is not optimal
- But how good is it?

Analyzing SimpleReversalSort (cont'd)

- But it can be sorted in two steps:

  π = 6 1 2 3 4 5

  - Step 1: 5 4 3 2 1 6
  - Step 2: 1 2 3 4 5 6
- So, SimpleReversalSort(π) is not optimal
- But how good is it?
- Optimal algorithms are **unknown** for many problems; approximation algorithms are used

Approximation Algorithms

- These algorithms find approximate solutions rather than optimal solutions
- The approximation ratio of an algorithm A on the problem with input π is:

$$A(\pi) / OPT(\pi)$$

where

$A(\pi)$ - solution produced by algorithm A

$OPT(\pi)$ - optimal solution of the problem

  - (in our case, π is an instance of the reversal sorting problem)

Approximation Algorithms

- If an algorithm has an approximation ratio = 1.5, it means that the solution it finds is never more than 150% of the optimal one.
  - For example, if the minimum sorting requires 10 reversals, the approx algorithm will use at most 15.

Approximation Ratio

- For algorithm A that minimizes objective function (minimization algorithm):
  - $\max_{|\pi| = n} A(\pi) / OPT(\pi)$
- For maximization algorithm:
  - $\min_{|\pi| = n} A(\pi) / OPT(\pi)$

Can we do better than SimpleReversalSort(π)?

Yes

Sometimes we need to characterize better the problem to make more sophisticated techniques

DISCLAIMER: DON'T BE AFRAID

You are requested to understand the general idea of greedy not how to elaborate this more sophisticated techniques

Adjacencies and Breakpoints

$$\pi = \pi_1 \pi_2 \pi_3 \ldots \pi_{n-1} \pi_n$$

- A pair of elements π_i and π_{i+1} are adjacent if

$$\pi_{i+1} = \pi_i \pm 1$$

- For example:

$$\pi = 1 \ \ 9 \ \ \underline{3 \ \ 4} \ \ \underline{7 \ \ 8} \ \ 2 \ \ \underline{6 \ \ 5}$$

- (3, 4) or (7, 8) and (6,5) are adjacent pairs

Breakpoints: An Example

There is a breakpoint between any adjacent element that are non-consecutive:

$$\pi = 1 \mid 9 \mid 3\ \ 4 \mid 7\ \ 8 \mid 2 \mid 6\ \ 5$$

- Pairs (1,9), (9,3), (4,7), (8,2) and (2,5) form breakpoints of permutation π

Breakpoints: An Example

There is a breakpoint between any adjacent element that are non-consecutive:

$$\pi = 1 \mid 9 \mid 3 \;\; 4 \mid 7 \;\; 8 \mid 2 \mid 6 \;\; 5$$

- Pairs (1,9), (9,3), (4,7), (8,2) and (2,5) form breakpoints of permutation π
- $b(\pi)$ - # breakpoints in permutation π

Extending Permutations

- We want to ensure that also the first and the last element are in the **right positions**. To do that…
- We put two elements $\pi_0 = 0$ and $\pi_{n+1} = n+1$ at the ends of π

Example:

$$\pi = 1 \mid 9 \mid 3\ \ 4 \mid 7\ \ 8 \mid 2 \mid 6\ \ 5$$



Extending with *0* and *10*

$$\pi = 0\ 1 \mid 9 \mid 3\ \ 4 \mid 7\ \ 8 \mid 2 \mid 6\ \ 5 \mid 10$$

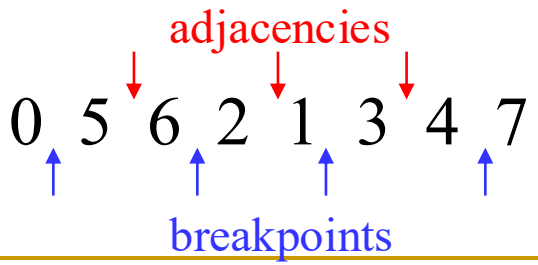
Note: A new breakpoint was created after extending

Sum up: Adjacency & Breakpoints

- An adjacency - a pair of adjacent elements that are consecutive
- A breakpoint - a pair of adjacent elements that are not consecutive

---

$\pi = 5\ \ 6\ \ 2\ \ 1\ \ 3\ \ 4 \longrightarrow$ Extend π with $\pi_0 = 0$ and $\pi_7 = 7$



Reversal Distance and Breakpoints

- Observation: each reversal eliminates at most 2 breakpoints.

π = | 2 3 | 1 | 4 | 6 5 |

0 | 2 | 3 1 | 4 | 6 5 | *7* $b(\pi) = 5$

0 1 | 3 2 | 4 | 6 5 | *7* $b(\pi) = 4$

0 1 2 3 4 | 6 5 | *7* $b(\pi) = 2$

0 1 2 3 4 5 6 *7* $b(\pi) = 0$

Reversal Distance and Breakpoints

- Observation: each reversal eliminates at most 2 breakpoints.
- This implies:

  reversal distance ≥ #breakpoints / 2

π = | 2 3 | 1 | 4 | 6 5 |

0 2 | 3 1 | 4 | 6 5 | *7* $b(\pi) = 5$

0 1 3 2 4 | 6 5 | *7* $b(\pi) = 4$

0 1 2 3 4 6 5 *7* $b(\pi) = 2$

0 1 2 3 4 5 6 *7* $b(\pi) = 0$

Sorting By Reversals: A Better Greedy Algorithm

BreakPointReversalSort(π)

1 **while** $b(\pi) > 0$

2 Among all possible reversals, choose reversal ρ minimizing $b(\pi)$ after its application

3 $\pi \leftarrow$ *apply* $\rho(i, j)$ *on* π

4 **return** π

Sorting By Reversals: A Better Greedy Algorithm

BreakPointReversalSort(π)

1 **while** $b(\pi) > 0$

2 Among all possible reversals, choose reversal ρ minimizing $b(\pi)$ after its application

3 $\pi \leftarrow apply\ \rho(i, j)\ on\ \pi$

4 **return** π

Problem: this algorithm may work forever (we cannot reduce the number of breakpoints anymore)

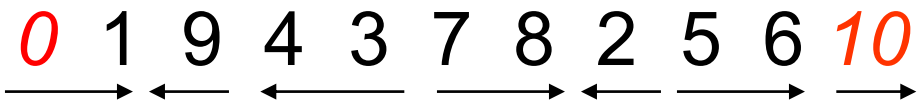
Can we do better than BreakPointReversalSort(π)?

Yes

We need to characterize even better the problem to exploit further aspects

Strips

- Strip: an interval between two consecutive breakpoints in a permutation
  - Decreasing strip: *strip* of elements in decreasing order (e.g. 6 5 and 3 2).
  - Increasing strip: *strip* of elements in increasing order (e.g. 7 8)



- A single-element strip can be declared either increasing or decreasing. We will choose to declare them as decreasing with exception of the strips with *0* and *n+1*

Reducing the Number of Breakpoints

Observation 1:

If permutation π contains at least one decreasing strip, then there exists a reversal ρ which decreases the number of breakpoints (i.e. $b(\pi)$ after $\rho < b(\pi)$)

Things To Consider

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | *9* $b(\pi) = 5$

  - Choose decreasing strip with the smallest element k in π (k = 2 in this case)

Things To Consider (cont'd)

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | *9* $b(\pi) = 5$

  - Choose decreasing strip with the smallest element k in π (k = 2 in this case)

Things To Consider (cont'd)

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | *9* $b(\pi) = 5$

  - Choose decreasing strip with the smallest element k in π ($k = 2$ in this case)
  - Find $k - 1$ in the permutation

Things To Consider (cont'd)

- For π = 1 4 6 5 7 8 3 2

  0 1 | 4 | 6 5 | 7 8 | 3 2 | 9 $b(\pi) = 5$

  - Choose decreasing strip with the smallest element *k* in π (*k* = 2 in this case)
  - Find *k* – *1* in the permutation
  - Reverse the segment between *k* and *k-1*:
  - 0 1 | 4 | 6 5 | 7 8 | 3 2 | 9 $b(\pi) = 5$

  - 0 1 2 3 | 8 7 | 5 6 | 4 | 9 $b(\pi) = 4$

Reducing the Number of Breakpoints Again

- If there is no decreasing strip, there may be no reversal ρ that reduces the number of breakpoints (i.e. $b(\pi)$ after $\rho \geq b(\pi)$ for any reversal ρ).
- By reversing an increasing strip (# of breakpoints stay unchanged), we will create a decreasing strip at the next step. Then the number of breakpoints will be reduced in the next step (observation 1).

Things To Consider (cont'd)

- There are no decreasing strips in $\boldsymbol{\pi}$, for:

Positions start at 1 (the added 0 is π_0): 3 and 4 are at positions 6 and 7

π = 0 1 2 | 5 6 7 | 3 4 | 8 $b(\pi) = 3$

$\rho(6,7)$ on π = 0 1 2 | 5 6 7 | 4 3 | 8 $b(\pi) = 3$

✓ $\rho(6,7)$ does not change the # of breakpoints

✓ $\rho(6,7)$ creates a decreasing strip thus guaranteeing that the next step will decrease the # of breakpoints by doing $\rho(3,7)$.

ImprovedBreakpointReversalSort

ImprovedBreakpointReversalSort(π)
1 **while** $b(\pi) > 0$
2 **if** π has a decreasing strip
3 Among all possible reversals, choose reversal ρ
that minimizes $b(\pi)$ *after* ρ
4 **else**
5 Choose a reversal ρ that flips an increasing strip in π
6 $\pi \leftarrow$ *apply* ρ *on* π
7 **return** π

ImprovedBreakpointReversalSort: performance?

- *ImprovedBreakPointReversalSort is the optimal solution?*

ImprovedBreakpointReversalSort: performance?

- *ImprovedBreakPointReversalSort is the optimal solution? Unfortunately, no..*
- *ImprovedBreakPointReversalSort* is an approximation algorithm
  - Optimal algorithm eliminates at most 2 breakpoints in every step: $d(\pi) \geq b(\pi) / 2$
  - It eliminates at least one breakpoint in every two steps; at most $2b(\pi)$ steps
  - Approximation ratio: $2b(\pi) / d(\pi)$

ImprovedBreakpointReversalSort: performance?

- *ImprovedBreakPointReversalSort is the optimal solution? Unfortunately, no..*
- *ImprovedBreakPointReversalSort* is an approximation algorithm
  - Approximation ratio: $2b(\pi) \ / \ d(\pi)$

- NOTE: we can compute $d(\pi)$ for a specific instance of π but not for all the instance of sorting by reversal problem, in this way we can compute the approximation ratio

Take home messages

When should we use Greedy Algorithms?

When should we use Greedy Algorithms?

- **Simple and easy to understand**

  we follow a simple idea: for every subproblem, a greedy algorithm tries to find the best optimal solution (e.g., in ImprovedBreakpointReversalSort, choose reversal ρ that minimizes b(π) after ρ)

When should we use Greedy Algorithms?

- **Simple and easy to understand**

  we follow a simple idea: for every subproblem, a greedy algorithm tries to find the best optimal solution (e.g., in ImprovedBreakpointReversalSort, choose reversal ρ that minimizes b(π) after ρ)

This is also the limit: following a simple rule (or rules) for **all** the subproblems might lead to **naive solutions** (i.e., solutions that consider the characteristics of every subproblem missing details)

When should we use Greedy Algorithms?

- **Can be used as a building block for other algorithms:** it can be used as a starting point for developing more complex algorithms.

We started with SimpleReversalSort
We improve it with BreakPointReversalSort
We improve it with ImprovedBreakpointReversalSort

When should we use Greedy Algorithms?

- **Fast and efficient** (compared to other techniques)

example: SimpleReversalSort runs in O(n) (n size of the vector π)

When should we use Greedy Algorithms?

- **Provides a good enough solution** (we have seen how good today for one problem)

ImprovedBreakPointReversalSort still finding acceptable **approximate** solutions (not the best ones)

We still don't know an **efficient algorithm** computing the **optimal solution** for sorting by reversal